

EE-2003 Computer Organization and Assembly Language



LAB 03

Addressing Modes: Data Declaration, Direct Addressing

Course Code: EE-2003

Semester: Fall 2025

Instructor: Engr. Muhammad Qasim

Email: Muhammad.qasim@nu.edu.pk

Department of Computer Science, National University of
Computer and Emerging Sciences FAST Peshawar

Objectives

After completing this lab, you'll be able to:

- Declare data variables of different sizes in memory.
 - Use labels to create symbolic names for memory addresses.
 - Understand and implement **Direct Addressing** to access memory variables.
 - Avoid and correct common **size mismatch errors**.
 - Write assembly programs that read from and write to memory.
-

Theory

In our previous work, we primarily used *immediate operands*, where the data is stored directly within the instruction itself (e.g., `mov ax, 5`). However, most real programs need to store and manipulate data in memory. This lab introduces the fundamental methods for declaring and accessing this data.

Data Declaration

To store data in memory, we first need to reserve space for it and optionally give it an initial value. Assembly language provides special instructions called **directives** for this purpose.

- **Labels:** To easily refer to a memory location, we can associate a symbolic name with it, called a **label**. A label is simply a name followed by a colon (:), placed on the line where you want to mark the address. The assembler automatically calculates the exact memory offset for each label.
- **Directives `db` and `dw`:**
 - `db` stands for "**define byte**" and reserves one byte (8 bits) of memory.
 - `dw` stands for "**define word**" and reserves one word (16 bits) of memory.
 - You can declare multiple values on a single line by separating them with commas.

Example:

`myByte: db 10` ; A single byte variable named myByte with value 10

`myWord: dw 5000` ; A single word variable named myWord with value 5000

`myArray: dw 5, 10, 15, 0` ; Four consecutive words starting at the address 'myArray'

Direct Addressing Mode

Direct addressing is a method where the memory address of the operand is fixed and specified directly within the instruction. To indicate that you want to access the *contents* of a memory location, you enclose its label in square brackets [].

- `mov ax, [myWord]` should be read as: "Move the contents of the memory location `myWord` into the AX register".
- You can also access memory locations by specifying a constant offset from a label, like `mov bx, [myArray+2]`. The assembler calculates this final address during assembly.

Important Rule: General instructions like `mov`, `add`, etc., **cannot** move data directly from one memory location to another. An instruction can have at most one operand in brackets. To move data between two memory variables, you must first load it into a register.

- **LEGAL:** `mov ax, [var1]`
- **LEGAL:** `mov [var2], ax`
- **ILLEGAL:** `mov [var2], [var1]`

Size Mismatch Errors

It's the **programmer's responsibility** to ensure that the size of the data being accessed matches the size of the register used. The assembler will not always catch these logical errors.

- If you declare data with `dw` (word), you must access it with a 16-bit register (e.g., AX, BX).
- If you declare data with `db` (byte), you must access it with an 8-bit register (e.g., AL, AH).

Sometimes, an instruction is ambiguous. For example, in `mov [myVar], 5`, the assembler doesn't know whether to treat 5 as a byte or a word. To resolve this, you must explicitly state the size using the byte or word specifiers.

`mov byte [myByteVar], 5` ; Stores 5 as a single byte

`mov word [myWordVar], 5` ; Stores 5 as a 16-bit word (0005h)

Example Codes

Example 1: Adding Three Numbers Using Memory

It declares three numbers as word-sized variables in memory, adds them, and stores the result back into another memory variable.

`[org 0x0100]`

`mov ax, [num1]` ; Load the value at memory location `num1` into `ax`

```
mov bx, [num2]    ; Load the value at memory location num2 into bx
add ax, bx        ; ax = ax + bx
mov bx, [num3]    ; Load the value at memory location num3 into bx
add ax, bx        ; ax = ax + bx (final sum)
mov [result], ax  ; Store the final sum from ax into the memory location 'result'
```

```
mov ax, 0x4c00    ; Terminate program
int 0x21
```

; --- Data Declarations ---

```
num1: dw 5
num2: dw 10
num3: dw 15
result: dw 0      ; A variable to store the result
```

Explanation: This code uses direct addressing to fetch each number from memory into a register before performing the addition. The final sum is then stored back in the result variable.

Example 2: Accessing an Array with Offsets

This program demonstrates accessing consecutive word-sized data using a single label and constant offsets, as shown in Example 2.2.

```
[org 0x0100]
```

```
mov ax, [numbers]    ; Load the first number (at offset 0)
add ax, [numbers+2]  ; Add the second number (at offset +2 bytes)
add ax, [numbers+4]  ; Add the third number (at offset +4 bytes)
mov [numbers+6], ax  ; Store the result in the fourth word (at offset +6 bytes)
```

```
mov ax, 0x4c00    ; Terminate program
int 0x21
```

; --- Data Declarations ---

numbers: dw 5, 10, 15, 0 ; An array of 4 words

Explanation: Since each variable is a word (2 bytes), the second number is at an offset of 2 bytes from the numbers label, the third is at an offset of 4 bytes, and so on.

Example 3: Handling Byte-Sized Data

This example modifies the previous one to work with byte-sized data, demonstrating the correct use of db and 8-bit registers like AL and BL

[org 0x0100]

```
mov al, [numbers]    ; Load the first number (byte) into al
add al, [numbers+1]  ; Add the second number (at offset +1 byte)
add al, [numbers+2]  ; Add the third number (at offset +2 bytes)
mov [numbers+3], al  ; Store the result in the fourth byte
```

```
mov ax, 0x4c00        ; Terminate program
int 0x21
```

; --- Data Declarations ---

numbers: db 5, 10, 15, 0 ; An array of 4 bytes

Explanation: Notice that the registers used are 8-bit (al) and the offsets are incremented by 1, as each data element is now a single byte.

Lab Tasks

Task 1 (5 Marks)

Write an assembly program that solves the equation $W = (X + Y) - Z$.

- In the data section, declare four **word-sized** variables: X, Y, Z, and W.
- Initialize X with the value 500, Y with 300, and Z with 150. Initialize W to 0.
- Use **direct addressing** to load the values of X, Y, and Z from memory into registers to perform the calculation.

- Store the final result back into the memory variable W.
- Verify your final result in the debugger by inspecting the memory location for W.

Task 2 (5 Marks)

Write an assembly program to swap the contents of a small data structure in memory.

- In the data section, declare the following structure:

Code snippet

```
studentId    dw 101
```

```
score1       db 95
```

```
score2       db 88
```

```
tempStorage  dw 0
```

- Your task is to swap the values of score1 and score2. You **must** use tempStorage to facilitate the swap.
- You must use **direct addressing** with correct register sizes (ax vs al) for each operation.
- After the program runs, the value at score1 should be 88, and the value at score2 should be 95.
- Verify the final state of score1 and score2 in the debugger.